C02 - AT3

Debugging and Optimization

Name :- Harshith G
Reg No :- 192525394
course :- ITA0615 :- Machine Learning

1) Debugging and optimization of a Multilayer Perceptron

A multilayer Perceptron (MLP) may experience slow convergence and get stuck in local minima due to vanishing gradients, unsuitable activation functions, poor weight initialization, or an inappropriate learning rate.

Debugging steps:

1. check gradients: monitor gradient values during backpropagation. very small gradients indicate the vanishing gradient problem.
2. check activation functions: Sigmoid and tanh may cause vanishing gradients in deep networks.
3. Check weight initialization: Use Xavier or He initialization to assign suitable initial weights.
4. Adjust the learning rate: A very low learning rate slows training, while a very high learning rate causes unstable updates. Test different learning rates.
5. Monitor training performance: Compare training and validation loss to identify convergence problems.

Optimization Techniques:

- Use adaptive optimizers such as Adam or Rmsprop to adjust learning rates.
- Apply batch normalization to stabilize activations and improve training.
- Use suitable activation functions and proper weight initialization.

2. Debugging and Optimization of a Genetic Algorithm:

(A) Genetic Algorithm may produce incosistent or suboptimal solutions because of incorrect chromosome encoding, a poorly designed fitness function, premature convergence, or insufficient population diversity

Debugging steps:

1. check chromosome encoding: Ensure that each chromosome correctly represents a possible solution.
2. Verify the fitness function: Confirm that it accurately measures solution quality.
3. monitor population density: check whether individuals become too similar in early generations
4. Inspect selection pressure: Excessive selection Pressure may cause premature convergence.
5. Test repeated runs: Compare results across multiple runs using the same settings to identity unstability.

Optimization Techniques:

- Mutation tuning: Adjust the mutation rate to maintain diversity and explore new solutions.
- Selection adjustment: Use tournament or rank-based selection to control selection pressure.
- Population size adjustment: Increase the population size if diversity is insufficient.
- Elitism: Preserve a few of the best individuals in each generations.

3. Deep Neural Network with Genetic Programming

A deep neural network combined with Genetic Programming (GP) may suffer from overfitting and high computational cost when the model is too complex, contains too many parameters, or is trained on insufficient data.

Debugging Steps:

1. Compare training and validation performance: High training accuracy with low validation accuracy indicates overfitting.
2. Check model complexity: Examine the number of layers, neurons and parameters.
3. Inspect the dataset: check whether the training data is insufficient, noisy, etc.

4. Analyze computational cost:
Measure training time and the time required to evaluate GP-generated solutions.

5. Evaluate generalization: Test the model on unseen data to determine whether it performs reliably.

Optimization Techniques:

- Regularization: Use L1 or L2 regularization to discourage unnecessarily complex models.
- Pruning: Remove unnecessary connections, neurons or redundant parts of the model.
- Early stopping: Stop training when validation performance no longer improves.

4. Debugging and optimization of an RNN for Time-Series Prediction.

A Recurrent Neural Network (RNN) may Produce inconsistent preditions and fail to capture long-term dependencies because of vanishing gradients, insufficient sequence length, or an unsuitable network architecture.

Debugging steps:

1. Check input sequences: Ensure that the time-series data is correctly ordered and contains sufficient historical information.
2. Inspect gradients: Identify whether gradients become too small during backpropagation through time.
3. Check preprocessing: Verify that the data is normalized and missing values are handled correctly.

Optimization Techniques:

- Use LSTM or GRU: These units use gating mechanisms to learn long-term dependencies more.
- Gradient clipping: Restrict excessively large gradients to improve training stability.
- Tune hyperparameters: Adjust the learning rate, hidden units and batch size.

5.) Debugging and optimization of a Naive Bayes Classifier

A Naive Bayes classifier may perform poorly on real-world text data because of noisy or irrelevant features, unsuitable prepoocessing, class imbalance or violations of the feature independence assumption.

Debugging steps:

1. Check text preprocessing: Remove unnecessary symbols and handle case conversion, tokenization, and missing values consistently.
2. Inspect features: Identify irrelevant or noisy words that reduce classification performance.
3. Check class distribution: Determine whether one class contains significantly more samples than others.

Optimization Techniques:

- Feature selection: Remove irrelevant features and retain informative words using methods such as chi-square selection.
- Smoothing: Apply laplace smoothing to handle words that do not appear in a particular class during training.
- Data balancing: Use oversampling, undersampling approaches when appropriate.
- Improve preprocessing: Apply suitable tokenization, Stop-word removal, and text normalization.